

Unit B3: Object-Oriented Programming (HL Extension)

Sub-topics: B3.2 Fundamentals of OOP for multiple classes

1. Syllabus Scope & Learning Outcomes

These topics are for Higher Level students only.

Syllabus Point	Command Term	Student Expectation (The "Do")
B3.2.1	Explain / Apply	Implement Inheritance using parent/child classes and super().
B3.2.2	Construct	Write code to demonstrate Polymorphism (overriding methods).
B3.2.3	Explain	Define Abstraction and implement Abstract Classes as blueprints.
B3.2.4	Explain	Differentiate between Composition ("death" relationship) and Aggregation (independent existence).
B3.2.5	Explain	Describe the purpose of Singleton, Factory, and Observer design patterns.

2. Key Terminology & Definitions

- **Inheritance:** A mechanism where a Child class (Subclass) derives attributes and methods from a Parent class (Superclass).
- **Polymorphism:** The ability of different classes to be treated as instances of the same general class through a common interface (e.g., different shapes having different area())

logic).

- **Overriding:** Writing a method in a Child class with the same name as the Parent class to replace the parent's behavior.
- **Abstract Class:** A class that cannot be instantiated; it serves only as a template for subclasses.
- **Composition:** A "Part-of" relationship where the child object *cannot* exist without the parent (e.g., House and Room).
- **Aggregation:** A "Has-a" relationship where the child object *can* exist independently (e.g., Library and Book).
- **Design Pattern:** A standard solution to a common programming problem.

3. Core Content & Exam Actions

B3.2.1: Inheritance

Theory:

- **IS-A Relationship:** A Dog *IS-A* Animal.
- **Syntax:** `class Child(Parent):`
- **Super:** Use `super().__init__()` to ensure the parent's attributes are initialized correctly before adding child-specific attributes.
- **Reusability:** Code is written once in the Parent and reused by all Children.

Student Exam Action:

- **Coding:** Given a Vehicle class, create a Car subclass that adds a `number_of_doors` attribute.

B3.2.2: Polymorphism

Theory:

- **Overriding:** Changing how a method works in the subclass.
- **Magic Methods:** Overriding `__str__` in Python to change how an object prints is a common form of polymorphism.
- **Flexibility:** You can store a list of different objects (e.g., `[Circle, Square, Triangle]`) and call `.area()` on all of them without knowing their specific type.

Student Exam Action:

- **Tracing:** Given a list of mixed objects, predict the output when a common method is called on each (e.g., `animal.speak()` prints "Woof" for Dog and "Meow" for Cat).

B3.2.3: Abstraction

Theory:

- **Abstract Base Class (ABC):** In Python, use `from abc import ABC, abstractmethod`.
- **Contract:** It forces subclasses to implement specific methods. If a subclass doesn't implement an abstract method, it creates an error.

- **Usage:** You never say `x = Shape()`. You say `x = Circle()`.

Student Exam Action:

- **Explanation:** Explain why Abstract classes are useful for teams of programmers (enforces a standard interface).

B3.2.4: Relationships (Composition vs Aggregation)

Theory:

- **Composition (Strong):** The "Container" controls the lifecycle. If you destroy the House object, the Room objects inside are destroyed. Represented by a **filled** diamond in UML.
- **Aggregation (Weak):** The "Container" holds a reference. If you destroy the Classroom object, the Student objects inside continue to exist. Represented by an **empty** diamond in UML.

Student Exam Action:

- **Scenario ID:** Given a scenario ("A Car has an Engine"), classify it as Composition or Aggregation and justify the choice based on lifecycle dependency.

B3.2.5: Design Patterns

Theory:

- **Singleton:** Ensures a class has only **one** instance (e.g., Database Connection, Game Settings).
- **Factory:** A method that creates objects for you, hiding the logic of instantiation (e.g., EnemyFactory creates different zombies based on level).
- **Observer:** A subscription model. When one object changes, it notifies all "observers" (e.g., YouTube channel notifying subscribers).

Student Exam Action:

- **Selection:** Given a problem description ("We need to ensure only one print spooler exists"), select the correct Design Pattern (Singleton).

4. Common Pitfalls & Examiner Tips

- **Pitfall (Composition vs Aggregation):** This is a high-mark exam question. The key is **lifecycle**. If the part dies when the whole dies, it's Composition. If the part survives, it's Aggregation.
- **Pitfall (Inheritance vs Composition):** Do not use Inheritance just to reuse code. Only use it for "IS-A" relationships. If it is a "HAS-A" relationship, use Composition/Aggregation.
- **Tip (Super):** In Python exams, forgetting `super().__init__(...)` in a subclass constructor is a common error that loses marks because the parent attributes are never set up.
- **Tip (Design Patterns):** You usually don't need to code these from scratch in an exam,

but you must be able to *explain* their purpose and *draw* a simple UML representation.